

A simple template for ISC documents

A. Lovelace, Prof. B. Liskov, Prof. N. Wirth

1 – Introduction

Ce document présente un aperçu des fonctionnalités offertes par [Typst](#), un système de composition typographique moderne. Les sections suivantes illustrent la mise en forme de texte, les listes, les tableaux, les images, le code source, les formules mathématiques et bien plus encore.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Quia ipsum suspendisse ultrices gravida. Risus commodo viverra maecenas accumsan.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Quia ipsum suspendisse ultrices gravida. Risus commodo viverra maecenas accumsan.

2 – Mise en forme du texte

Typst offre une syntaxe simple et intuitive pour formater du texte. On peut écrire en **gras**, en *italique*, ou en ***gras italique***. Il est aussi possible d'utiliser du `code en ligne` directement dans le texte.

Les [couleurs](#) sont facilement applicables, tout comme les changements de **taille** ou de *police*. On peut également souligner, ~~barrer~~ ou mettre en PETITES CAPITALES.

The best programs are written so that computing machines can perform them quickly and so that human beings can understand them clearly.

— Donald Knuth

2.1 – Listes

Les listes à puces et numérotées sont très simples à créer :

- Premier élément
- Deuxième élément avec sous-éléments :
 - Sous-élément A
 - Sous-élément B
- Troisième élément

Et les listes numérotées :

- 1. Analyser le problème
- 2. Concevoir une solution
- 3. Implémenter et tester
- 4. Documenter le résultat

3 – Tableaux et figures

3.1 – Tableaux

Les tableaux permettent de présenter des données de manière structurée. Voici un comparatif de quelques langages de programmation :

Langage	Paradigme	Typage	Année
Python	Multi-paradigme	Dynamique	1991
Scala	Fonctionnel / OO	Statique	2004
Rust	Système	Statique	2010
Typst	Markup / Script	Dynamique	2023
C	Procédural	Statique	1972

Table 1 - Comparaison de langages de programmation

Comme le montre la Table 1, chaque langage a ses propres caractéristiques. On peut référencer les tableaux et figures automatiquement grâce aux labels.

3.2 – Insertion d’images

Les images s’insèrent facilement avec la fonction `image` . Voici un exemple :



Figure 1 - Une image d’exemple insérée dans le document

La Figure 1 montre comment inclure une image avec une légende. Les images peuvent être redimensionnées, alignées et référencées dans le texte.

4 – Code et mathématiques

4.1 – Blocs de code

Typst peut afficher du code source avec coloration syntaxique :

```
1 def fibonacci(n: int) → list[int]:
```

```

2  """Génère les n premiers nombres de Fibonacci."""
3  fib = [0, 1]
4  for i in range(2, n):
5      fib.append(fib[-1] + fib[-2])
6  return fib
7
8  # Afficher les 10 premiers
9  print(fibonacci(10))

```

Et un autre exemple en Scala :

```

1  case class Student(name: String, grade: Double)
2
3  val students = List(
4      Student("Alice", 5.5),
5      Student("Bob", 4.8),
6      Student("Charlie", 5.9),
7  )
8
9  val average = students.map(_.grade).sum / students.length
10 println(f"Moyenne: $average%.1f")

```

4.2 – Formules mathématiques

Typst dispose d'un excellent support des mathématiques. Par exemple, la formule d'Euler :

$$e^{i\pi} + 1 = 0$$

Une intégrale classique :

$$\int_0^\infty e^{-x^2} dx = \frac{\sqrt{\pi}}{2}$$

Ou encore la définition d'une matrice et d'un système d'équations :

$$A = \begin{pmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,n} \\ a_{2,1} & a_{2,2} & \cdots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m,1} & a_{m,2} & \cdots & a_{m,n} \end{pmatrix}$$

Les formules en ligne comme $\sum_{k=1}^n k = \frac{n(n+1)}{2}$ s'intègrent naturellement dans le texte.

4.3 – Boîtes et encadrés

On peut utiliser les boîtes de `showybox` pour mettre en valeur du contenu :

TODO Compléter cette section avec d'autres exemples

5 – Citer ses sources

Il est important de citer les sources que l'on utilise. Par exemple, les deux travaux [1], [2] et [3] sont des papiers très intéressants à lire et dont les références complètes se trouvent dans la bibliographie à la fin de ce document.

Bibliographie

- [1] P.-A. Mudry et G. Tempesti, « Self-Scaling Stream Processing: A Bio-Inspired Approach to Resource Allocation through Dynamic Task Replication », in *2009 NASA/ESA Conference on Adaptive Hardware and Systems*, 2009, p. 353-360. doi: [10.1109/AHS.2009.25](https://doi.org/10.1109/AHS.2009.25).
- [2] P.-A. Mudry, G. Zufferey, et G. Tempesti, « A hybrid genetic algorithm for constrained hardware-software partitioning », in *2006 IEEE Design and Diagnostics of Electronic Circuits and systems*, 2006, p. 1-6. doi: [10.1109/DDECS.2006.1649561](https://doi.org/10.1109/DDECS.2006.1649561).
- [3] P.-A. Mudry, *A hardware-software codesign framework for cellular computing*. Lausanne: EPFL, 2009, p. 287. doi: [10.5075/epfl-thesis-4354](https://doi.org/10.5075/epfl-thesis-4354).
- [4] L. Masinter, « RFC 2324: Hyper Text Coffee Pot Control Protocol (HTCPCP/1.0) ». Consulté le: 29 mai 2026. [En ligne]. Disponible sur: <https://www.rfc-editor.org/rfc/rfc2324>
- [5] M. LLC, « MS Windows NT Kernel Description ». Consulté le: 30 septembre 2010. [En ligne]. Disponible sur: <http://web.archive.org/web/20080207010024/http://www.808multimedia.com/winnt/kernel.htm>